

NAVA DeepStream Pipeline

Bug & Investigation Reference

Version 2.0 | June 2026

Covers 25 bugs across DeepStream core, Backend, Kafka, and newly identified cascade-failure root causes

25 Total Bugs	2 New (Critical)	23 Resolved	2 In Progress
---------------	------------------	-------------	---------------

SECTION 01

DeepStream / Pipeline

*GStreamer topology, hot-reload, recovery, evidence — Bugs 01–08 + NEW 24–25***BUG
01**

DeepStream Hot-Reload Breaking — Runtime Camera Add/Remove Fails

STATUS **RESOLVED** SEVERITY **HIGH** COMPONENT [DeepStream core / GStreamer](#)

SYMPTOM Adding or removing cameras at runtime caused pipeline errors or silently failed with no recovery.

ROOT CAUSE Hot-reload operations ran on a background Kafka thread instead of the GLib main thread. `source_id` used a monotonic counter and eventually collided with `nvstreammux` batch slot indices. Remove and add operations were not serialised.RESOLUTION `_hot_reload_queue` + 50ms GLib timer routes all ops to the main thread. `_allocate_source_id()` finds the lowest free slot. `_hot_reload_ops_lock` serialises execution. `DS_HOT_RELOAD_DELAY_MS` gives `nmultiuriscbin` time to complete async state transitions before re-add.CODE REF `_process_hot_reload_queue()`, `_allocate_source_id()`, `_hot_reload_ops_lock`, `DS_HOT_RELOAD_DELAY_MS` env**BUG
02**

'DeepStreamApp' Object Has No Attribute '_anpr' on Stream Remove

STATUS **RESOLVED** SEVERITY **MEDIUM** COMPONENT [DeepStream core](#)SYMPTOM `AttributeError` crash during camera removal when ANPR feature was not wired into the pipeline.ROOT CAUSE `_anpr` was never initialised in `__init__`. The remove path called `self._anpr.clear_source()` unconditionally.RESOLUTION Added `self._anpr = None` in `__init__`. All access sites changed to `getattr(self, '_anpr', None)` before calling `clear/remove`.CODE REF `__init__`: `self._anpr = None`; `_ds_remove_stream`: `getattr(self, '_anpr', None)`**BUG
03**

Heartbeat Thread Blocking Main Loop (~120s Delays, Delayed OFFLINE)

STATUS **RESOLVED** SEVERITY **HIGH** COMPONENT [Heartbeat / Threading](#)

SYMPTOM Camera OFFLINE status was delayed by up to 120 seconds. Heartbeat publishes were slow and irregular.

ROOT CAUSE `_heartbeat_loop` called `_run_on_main_loop()` for recovery logic before publishing heartbeats. When the main loop was busy processing a remove/add cycle, this blocked the heartbeat thread for the entire duration.

RESOLUTION	Publish heartbeats first unconditionally. Enqueue recovery ops via <code>_enqueue_hot_reload()</code> which is non-blocking — it appends to the queue and returns immediately.
CODE REF	<code>_heartbeat_loop()</code> , <code>_enqueue_hot_reload(_heartbeat_pipeline_ops)</code>

BUG 04	All Cameras Go OFFLINE Together — Mass Cascade Failure at 19 Cameras		
	STATUS	RESOLVED	SEVERITY CRITICAL COMPONENT DeepStream core / Recovery
	SYMPTOM	One stale RTSP camera triggered recovery. Within 45 seconds, all 15 active cameras went OFFLINE simultaneously. Pipeline stalled for 141+ seconds with no auto-recovery.	
	ROOT CAUSE	Stream recovery (REMOVE on <code>nvmultiuriscbin</code>) blocked the shared batch pipeline's tracker probe for all cameras. Every camera's <code>last_frame_at</code> aged out together after 30s TTL, giving the appearance of simultaneous failure. No per-probe liveness detection existed.	
	RESOLUTION	PIPELINE_STALL detection via <code>_last_probe_at</code> timestamp. Full pipeline restart on stall. Per-camera remove+add only when probe is healthy AND exactly one stream is stale. Re-add watchdog (30s timeout). Process exit after 180s unrecoverable stall with Docker auto-restart.	
	CODE REF	<code>_last_probe_at</code> , <code>_is_pipeline_stalled()</code> , <code>_recover_stale_streams()</code> , <code>_pipeline_stall_exit_seconds</code> , <code>MASS_FAILURE_THRESHOLD_PCT</code>	

BUG 05	NAVA8 Removed But Never Re-Added After Recovery		
	STATUS	RESOLVED	SEVERITY HIGH COMPONENT DeepStream core / Recovery
	SYMPTOM	After stream recovery removed NAVA8, the camera was permanently absent from the pipeline. No automatic re-add occurred.	
	ROOT CAUSE	The delayed <code>Glib.timeout_add</code> re-add callback never ran because the main loop was frozen at that moment. No watchdog monitored whether the re-add completed.	
	RESOLUTION	<code>_pending_recovery_adds</code> dict tracks in-flight recoveries with a deadline. Watchdog in <code>_check_pending_recovery_adds()</code> fires on each heartbeat tick. <code>_force_add=True</code> bypasses the duplicate-camera guard on recovery re-add. Timeout triggers full pipeline restart.	
	CODE REF	<code>_pending_recovery_adds</code> , <code>_check_pending_recovery_adds()</code> , <code>_ds_add_stream(_force_add=True)</code>	

BUG 06	Evidence Failing at 15+ Cameras — No JPEG for <code>source_id</code>		
	STATUS	RESOLVED	SEVERITY HIGH COMPONENT Evidence / Snapshot branch
SYMPTOM	Events were emitted but evidence images were missing: 'evidence images skipped for UNAUTHORIZED_ENTRY (no JPEG for <code>source_id=N</code>)' logged for multiple cameras simultaneously.		

ROOT CAUSE	On-demand evidence capture could not reliably acquire frames at scale. The continuous JPEG cache had been removed in an earlier refactor, leaving no reliable frame source at event time.
RESOLUTION	Reverted to continuous tee -> nvstreamdemux -> nvvideoconvert -> nvjpegenc -> appsink per camera. Tuned: EVIDENCE_JPEG_RETRIES=20, EVIDENCE_JPEG_RETRY_MS=300, EVIDENCE_FRAME_CACHE_MAX_AGE_SECONDS=120, DS_TARGET_FPS=5.
CODE REF	<code>_ensure_snapshot_cache_branch()</code> , <code>_on_new_jpeg_sample()</code> , <code>_get_jpeg_for_evidence()</code> , <code>appsink per source_id</code>

BUG 07	worker_alive Always True While Pipeline Was Dead STATUS RESOLVED SEVERITY MEDIUM COMPONENT Heartbeat
SYMPTOM	Heartbeat payloads reported worker_alive: true even when the tracker probe had been silent for minutes.
ROOT CAUSE	worker_alive was hardcoded to True in the heartbeat payload regardless of actual pipeline state.
RESOLUTION	<code>_pipeline_worker_alive()</code> method: returns False when <code>_is_pipeline_stalled()</code> is True or pipeline is None with cameras present.
CODE REF	<code>_pipeline_worker_alive()</code> , <code>_is_pipeline_stalled()</code>

BUG 08	LOAD_SUMMARY Streaming Sources Count Misleading STATUS RESOLVED SEVERITY LOW COMPONENT Diagnostics
SYMPTOM	LOAD_SUMMARY logged live_streams that included cameras that had not received frames in hours.
ROOT CAUSE	Count used <code>len(_last_frame_at)</code> which tracks every source ever seen, not sources active within the TTL window.
RESOLUTION	live_streams now counts only sources with <code>last_frame_at</code> within <code>STREAM_TTL_SECONDS</code> . Added <code>probe_age_sec</code> , <code>stalled bool</code> , and <code>recovery_in_flight</code> to LOAD_SUMMARY.
CODE REF	<code>_count_live_streaming_sources()</code> , <code>_log_load_summary()</code>

SECTION 02

Backend / Heartbeats / Camera Health*SSE, datetime handling, MongoDB, TTL, system_logs — Bugs 09–15*

BUG 09	SSE Stream Crash — TypeError: datetime Not JSON Serializable
-----------	---

	STATUS RESOLVED SEVERITY MEDIUM COMPONENT Backend / SSE
SYMPTOM	Server-Sent Events stream crashed with TypeError when datetime objects were included in payloads.
ROOT CAUSE	json.dumps() called on SSE payloads containing Python datetime objects without a custom serializer.
RESOLUTION	json.dumps(..., default=str) in sse.py converts datetime and other non-serializable types to strings.
CODE REF	sse.py: json.dumps(payload, default=str)

BUG 10	Camera Health Datetime Crash — Naive vs Timezone-Aware Subtraction
	STATUS RESOLVED SEVERITY MEDIUM COMPONENT Backend / Camera health
SYMPTOM	TypeError: can't subtract offset-naive and offset-aware datetimes in camera health calculations.
ROOT CAUSE	MongoDB timestamps were sometimes stored as timezone-naive, sometimes as timezone-aware datetimes. Python subtraction between the two types raises TypeError.
RESOLUTION	_parse_ts() and _parse_timestamp() helper functions normalise all timestamps to UTC-aware before arithmetic. Applied in camera_health.py, store.py, and heartbeat_consumer.py.
CODE REF	camera_health.py: _parse_ts(); store.py, heartbeat_consumer.py: _parse_timestamp()

BUG 11	MongoDB offline_since Update Conflict — \$set and \$unset Collision
	STATUS RESOLVED SEVERITY MEDIUM COMPONENT Backend / MongoDB
SYMPTOM	MongoDB write errors when updating camera status: conflicting updates on field 'offline_since'.
ROOT CAUSE	The same Mongo update operation included both \$set: {offline_since: ...} and \$unset: {offline_since: ""} on the same field, which MongoDB rejects.
RESOLUTION	offline_since is omitted from \$set when \$unset is present in the same operation in camera_health.py.
CODE REF	camera_health.py: omit offline_since from \$set when \$unset is used

BUG 12	False OFFLINE / TTL Mismatch — Pipeline 30s vs Backend 60s
	STATUS RESOLVED SEVERITY HIGH COMPONENT Backend / Heartbeat
SYMPTOM	Cameras appeared OFFLINE in the UI seconds after going ONLINE. Backend declared cameras stale before the next heartbeat arrived.

ROOT CAUSE	DeepStream published heartbeats every 30s but the backend hardcoded HEARTBEAT_TTL_SECONDS=60. Under any Kafka latency or heartbeat interval variance, the backend would declare cameras stale.
RESOLUTION	HEARTBEAT_TTL_SECONDS made configurable (default 120s) in config.py and docker-compose.yml. Stale check uses settings.HEARTBEAT_TTL_SECONDS throughout.
CODE REF	config.py: HEARTBEAT_TTL_SECONDS=120; heartbeat_consumer.py: stale threshold from settings

BUG 13	Stale Mongo Cameras Not Logged in system_logs		
	STATUS	RESOLVED	SEVERITY LOW COMPONENT Backend / Logging
	SYMPTOM	Cameras that went stale via MongoDB timeout were marked OFFLINE but no system_log entry was created.	
	ROOT CAUSE	The Mongo-stale code path called mark_offline() but skipped mark_offline_no_heartbeat() which writes system_logs.	
	RESOLUTION	mark_offline_no_heartbeat() called for both tracker-stale and Mongo-stale code paths.	
	CODE REF	camera_health.py: mark_offline_no_heartbeat() for Mongo-stale path	

BUG 14	Heartbeat Consumer Used datetime.now() Instead of Message Timestamp		
	STATUS	RESOLVED	SEVERITY MEDIUM COMPONENT Backend / Heartbeat consumer
	SYMPTOM	last_heartbeat in MongoDB reflected consumer processing time, not the actual time the heartbeat was sent.	
	ROOT CAUSE	heartbeat_consumer.py used datetime.now(UTC) for last_heartbeat instead of the timestamp embedded in the Kafka message payload.	
	RESOLUTION	Parse and store the heartbeat message's own timestamp field via _parse_timestamp().	
	CODE REF	heartbeat_consumer.py: _parse_timestamp(msg.value['timestamp'])	

BUG 15	Heartbeat Consumer Silent Drops / Bad Camera Lookup		
	STATUS	RESOLVED	SEVERITY MEDIUM COMPONENT Backend / Heartbeat consumer
	SYMPTOM	Valid heartbeat messages were silently dropped. Camera status updates did not reflect in the database.	
	ROOT CAUSE	Invalid payloads were dropped with no log. Camera lookup used ObjectId(camera_id) only, which failed for cameras identified by UI camera_id string rather than Mongo_id.	
	RESOLUTION	Warning logs added for bad/invalid payloads. resolve_camera() helper tries both Mongo_id and UI camera_id field for lookup.	
	CODE REF	heartbeat_consumer.py: resolve_camera(), warning logs on bad payloads	

SECTION 03

Kafka Producer / Consumer

*Producer races, silent drops, offset commits, threading — Bugs 16–23***BUG
16**

One Failed Heartbeat Publish Aborted All Remaining Cameras

STATUS **RESOLVED** SEVERITY **HIGH** COMPONENT [Kafka / Heartbeat](#)

SYMPTOM

If one camera's Kafka heartbeat send failed, no subsequent cameras in the same batch received heartbeat publishes.

ROOT
CAUSE

`_publish_heartbeats` used return on first Kafka send exception, abandoning the loop over all cameras.

RESOLUTION

Changed to continue per-camera on failure. Producer reset only when all cameras in a batch fail.

CODE REF

`_publish_heartbeats()`: continue per camera; producer reset only on total failure

**BUG
17**

Heartbeat Producer Race Condition — Lock Unused on Create/Send

STATUS **RESOLVED** SEVERITY **HIGH** COMPONENT [Kafka / Threading](#)

SYMPTOM

Intermittent `NoneType` errors and duplicate producer creation under concurrent heartbeat and shutdown paths.

ROOT
CAUSE

`_heartbeat_lock` was only acquired during `stop()`. Producer creation and `send()` calls were unprotected, allowing two threads to create producers simultaneously.

RESOLUTION

All producer access (create, send, flush, close) wrapped under `_heartbeat_lock`. `acks='all'` for durability.

CODE REF

`_heartbeat_lock` held during `_ensure_heartbeat_producer()`, `send`, `flush`, and `close`

**BUG
18**

Camera Publish to Kafka Failed Silently

STATUS **RESOLVED** SEVERITY **MEDIUM** COMPONENT [Kafka](#)

SYMPTOM

Camera state changes were not reflected in Kafka-consuming services. No error in logs.

ROOT
CAUSE

`publish_camera_event()` returned silently when the producer was down or when `send()` returned a failure future.

RESOLUTION

Warning and error logs added in `kafka_bus.py`. `send()` return value explicitly checked.

CODE REF

`kafka_bus.py`: warning on unavailable producer; check `send()` return value

**BUG
19****Event Emitter Failed Silently When Kafka Producer Unavailable**STATUS **RESOLVED** SEVERITY **MEDIUM** COMPONENT [Kafka / Event emit](#)

SYMPTOM Events were classified and processed but never appeared in downstream consumers. No error visible in logs.

ROOT CAUSE send() returned False when Kafka producer was unavailable, with no log at the call site.

RESOLUTION Warning logged when producer unavailable. Richer exception context on send failure in event_emitter.

CODE REF event_emitter.py: warning on unavailable producer; exception context on send failure

**BUG
20****Kafka Offset Not Committed on Heartbeat Consumer Shutdown**STATUS **RESOLVED** SEVERITY **MEDIUM** COMPONENT [Kafka / Consumer](#)

SYMPTOM On restart after graceful shutdown, heartbeat consumer reprocessed the last batch of messages.

ROOT CAUSE Batch commit every 10 messages; no final consumer.commit() call in the shutdown finally block.

RESOLUTION Final consumer.commit() before stop() when there are uncommitted messages in the current batch.

CODE REF heartbeat_consumer.py: consumer.commit() in finally block

**BUG
21****Kafka Bus Producer start() Uncaught Exception on Broker Down**STATUS **RESOLVED** SEVERITY **MEDIUM** COMPONENT [Kafka](#)

SYMPTOM Entire bus initialisation crashed with an unhandled exception when Kafka broker was unavailable at startup.

ROOT CAUSE producer.start() had no try/except wrapper. Broker unavailability at init time propagated as an unhandled exception.

RESOLUTION try/except around producer.start() with error logging. Application continues with degraded Kafka capability.

CODE REF kafka_bus.py: try/except around producer.start()

**BUG
22****Camera Hot-Reload UI Update on Wrong Thread — GStreamer Unsafe**STATUS **RESOLVED** SEVERITY **HIGH** COMPONENT [DeepStream core / Threading](#)

SYMPTOM	Camera annotation and event-type updates from the UI caused intermittent classifier state corruption and occasional pipeline errors.
ROOT CAUSE	<code>_sync_camera_after_ui_update()</code> ran directly on the Kafka watcher thread, mutating classifier state concurrently with the GStreamer streaming thread running the tracker probe.
RESOLUTION	UI update ops enqueued via <code>_enqueue_hot_reload()</code> to run on the GLib main thread.
CODE REF	<code>_camera_watcher_loop: _enqueue_hot_reload(_on_sync);</code> <code>_sync_camera_after_ui_update</code> on main thread

BUG 23	Malformed nava.cameras JSON Could Crash Camera Watcher STATUS RESOLVED SEVERITY MEDIUM COMPONENT Kafka / Camera watcher
SYMPTOM	A single malformed Kafka message on the cameras topic caused the watcher thread to crash and stop processing all subsequent camera updates.
ROOT CAUSE	<code>json.loads</code> in the Kafka value deserializer had no error handling. A JSON parse error propagated and crashed the consumer loop.
RESOLUTION	<code>_safe_kafka_camera_value()</code> wrapper catches all parse errors, logs a warning, and returns <code>None</code> so the message is skipped cleanly.
CODE REF	<code>_safe_kafka_camera_value()</code> ; <code>_camera_watcher_loop</code> skips <code>None</code> values

SECTION 04

Newly Identified — Critical Investigation

Root-cause findings from June 2026 log analysis and code review

BUG 24 NEW	Blocking I/O on GLib Main Thread Causes Cascade Camera Failure STATUS IN PROGRESS SEVERITY CRITICAL COMPONENT DeepStream core / Recovery
SYMPTOM	One stale RTSP camera triggers single-camera recovery. The GLib main thread is blocked for up to 13.5 seconds. During this window, all other cameras' <code>ASYNC_DONE</code> bus messages pile up unprocessed. <code>nvmultiurisrcbin</code> 's internal watchdog fires for all cameras simultaneously. 15+ cameras go OFFLINE within seconds of each other (confirmed in logs: 2026-06-08 12:33:07, all cameras show <code>last_frame_age_sec ~48s</code>).
ROOT CAUSE	Five blocking operations run sequentially on the GLib main thread inside <code>_ds_remove_stream</code> and <code>_restart_stream</code> : (1) <code>httpx.post('/remove')</code> blocks up to 5s, (2) <code>time.sleep(drain_sec)</code> blocks 0.5s, (3) <code>_wait_for_source_removal()</code> polling loop blocks up to 2s, (4) <code>_verify_rtsp_reachable()</code> TCP connect in <code>GLib.timeout_add</code> callback blocks up to 3s, (5) <code>httpx.post('/add')</code> blocks up to 5s. GStreamer requires the main loop to iterate continuously to dispatch <code>ASYNC_DONE</code> — the deadlock between <code>httpx</code> waiting for the HTTP 200 and <code>nvmultiurisrcbin</code> waiting for the main loop to process <code>ASYNC_DONE</code> is the core failure. Additionally, <code>_pipeline_restart_cooldown=300s</code> silently blocked stall recovery for 5 minutes after any prior restart attempt.

RESOLUTION	DESIGNED — NOT YET IMPLEMENTED. All blocking I/O moves to a dedicated <code>_remove_add_worker</code> thread (<code>threading.Thread</code>). Main thread only spawns worker and returns immediately to GLib loop. GStreamer state ops (<code>_teardown_source_state</code> , <code>_register_added_source</code>) posted back to main thread via <code>GLib.idle_add()</code> . <code>threading.Event</code> handshake ensures teardown completes before re-add REST call. <code>_pipeline_restart_cooldown</code> bypassed when <code>live_streams == 0</code> (total blackout). Prometheus counter added for cooldown-skipped restarts.
CODE REF	<code>_restart_stream()</code> , <code>_do_remove()</code> , <code>_wait_for_source_removal()</code> , <code>_verify_rtsp_reachable()</code> — all to be refactored to <code>_remove_add_worker</code> thread + <code>GLib.idle_add()</code> bridge

BUG 25 NEW	Incomplete State Cleanup in <code>_ds_remove_stream</code> Causes JPEG Miss on Re-Add		
	STATUS	IN PROGRESS	SEVERITY HIGH COMPONENT DeepStream core / Snapshot branch
	SYMPTOM	After camera recovery (remove + re-add), evidence JPEGs are missing: 'no JPEG for source_id=N'. Confirmed for source_id=0, 6, 8 in production logs (2026-06-08 12:32). Classifiers on re-added cameras have stale gate line state. Heartbeat status logs show wrong OFFLINE->ONLINE transitions on first re-add frame.	
	ROOT CAUSE	State cleanup is scattered across two blocks ('Bug 1 fix' pre-remove and 'Bug 6 fix' post-remove) and misses five critical items: (1) <code>_snapshot_branch_source_ids</code> never cleared — <code>_ensure_snapshot_cache_branch</code> skips re-creation on re-add because source_id still appears registered, leaving the appsink dead; (2) GStreamer snapshot branch elements (<code>q_snap_N</code> , <code>nvvideoconvert</code> , <code>capsfilter</code> , <code>nvjpegenc</code> , <code>appsink</code>) never removed from pipeline — element leak per recovery cycle; (3) <code>nvstreamdemux</code> request pad never released — pad leak per remove; (4) <code>_last_published_stream_status</code> (keyed by <code>cam_oid</code> , not <code>source_id</code>) never purged — stale OFFLINE status on re-add; (5) <code>_frame_counter</code> not cleared atomically with <code>_last_frame_at</code> .	
	RESOLUTION	DESIGNED — NOT YET IMPLEMENTED. Two new consolidated methods: <code>_teardown_snapshot_branch(source_id)</code> sets elements to NULL, removes from pipeline, releases demux pad, discards from <code>_snapshot_branch_source_ids</code> . <code>_teardown_source_state(source_id, cam_oid)</code> single authoritative cleanup calling <code>teardown_snapshot_branch</code> plus clearing all 11 state maps under their locks in correct order. All scattered Bug 1 + Bug 6 blocks replaced by single <code>_teardown_source_state()</code> call.	
	CODE REF	<code>_ds_remove_stream()</code> Bug1/Bug6 blocks → <code>_teardown_source_state()</code> + <code>_teardown_snapshot_branch()</code> ; <code>_snapshot_branch_source_ids.discard()</code>	

SECTION 05

Architecture Reference

GStreamer plugins, thread map, and lock inventory

GStreamer Plugin Inventory (11 plugins)

PLUGIN	PURPOSE	THREAD
nvmultiursrcbin	Entry point. RTSP decode via NVDEC, per-source nvuridecodebin, nvstreammux batching, REST :9000 for runtime add/remove.	GStreamer streaming threads (internal)
nvinfer	YOLOX TensorRT FP16 batch inference on the full camera batch.	GStreamer streaming threads
nvtracker (NvDCF)	Persistent object_id (track_id) across frames via NvDCF algorithm.	GStreamer streaming threads
tee	Splits post-tracker batch into main branch and snapshot branch simultaneously.	GStreamer streaming threads
queue (x N+2)	q_main, q_snap_root, and one q_snap_N per source. Decouples pipeline stages. q_snap variants are leaky (drop old frames) to prevent back-pressure.	GStreamer streaming threads
nvstreamdemux	Splits batched buffer back into per-source streams using source_id in frame metadata. One src_N pad per camera.	GStreamer streaming threads
nvvideoconvert	Converts NVMM GPU buffer to I420 format required by nvjpegenc. One instance per camera.	GStreamer streaming threads
capsfilter	Enforces video/x-raw(memory:NVMM),format=I420 caps negotiation. One per camera.	GStreamer streaming threads
nvjpegenc	GPU JPEG encoding at quality=80. One per camera.	GStreamer streaming threads
appsink	GStreamer-to-Python boundary. Emits new-sample signal → <code>_on_new_jpeg_sample()</code> callback. One per camera.	GStreamer streaming thread (callback)
fakesink	Terminates the main (metadata-only) branch. sync=0 async=0 to avoid pipeline throttling.	GStreamer streaming threads

Thread Map

THREAD	RUNS	RULES / CONSTRAINTS
GLib main thread	GStreamer pipeline ops, element state, pad link/unlink, bus message dispatch, hot-reload queue drain (50ms timer), <code>_on_bus_message</code>	All GStreamer element and state operations MUST run here. Never block.

THREAD	RUNS	RULES / CONSTRAINTS
GStreamer streaming threads (internal)	<code>_tracker_probe</code> (per batch), <code>_on_new_jpeg_sample</code> (appsink callback)	GStreamer-managed. Write to <code>_latest_jpeg</code> and <code>_last_frame_at</code> under locks. Do not call any Gst API that modifies pipeline state.
ds-heartbeat	<code>_heartbeat_loop</code> , <code>_publish_heartbeats</code> (Kafka send), <code>_enqueue_hot_reload(_heartbeat_pipeline_ops)</code>	Reads <code>_last_frame_at</code> under lock. Enqueues GStreamer ops, never calls them directly.
ds-camera-watcher	Kafka consumer loop, <code>_enqueue_hot_reload</code> on camera add/remove/update	Never touches pipeline or classifiers directly. All ops enqueued.
thumb-N (per camera)	<code>api.upload_camera_thumbnail</code> — one-shot HTTP upload	Daemon thread, fires once on first JPEG. No shared state.
remove-add worker (NEW)	<code>_remove_add_worker</code> : <code>httpx.post(/remove)</code> , <code>sleep</code> , <code>httpx.post(/add)</code> , <code>_verify_rtsp_reachable</code> , <code>api.get_camera</code>	All blocking I/O. Posts state ops back to main thread via <code>GLib.idle_add()</code> .
Prometheus HTTP server (internal)	Serves <code>/metrics</code> endpoint for Grafana scrape	Prometheus client manages its own thread. Metric calls (<code>record_frame</code> etc.) are thread-safe via client library locks.

Lock Inventory

- `_latest_jpeg_lock` — guards `_latest_jpeg` dict; written by appsink streaming thread, read by probe and heartbeat thread
- `_last_frame_lock` — guards `_last_frame_at` and `_frame_counter`; written by probe (streaming thread), read by heartbeat thread
- `_heartbeat_lock` — guards `_heartbeat_producer` reference; prevents concurrent create/send/close from heartbeat and main threads
- `_source_id_lock` — guards slot allocator; prevents two concurrent adds from claiming the same `source_id`
- `_hot_reload_lock` — guards `_hot_reload_depth` counter; tracks in-flight add/remove sessions for EOS suppression
- `_hot_reload_ops_lock` — serialises execution of hot-reload queue callbacks on the main thread
- `_recovery_lock` — prevents two concurrent per-camera recovery attempts from both passing the `in_flight` guard
- `_thumbnail_lock` — atomic check-and-set for `_thumbnail_saved`; prevents duplicate upload threads per camera